

11-AI-ARCHITECTURE-RULES.md

Version : V1.0

Statut : ACTIVE

Catégorie : AI Engineering Framework

Projet : Vitala

1. Objectif

AI-ARCHITECTURE-RULES.md définit les règles obligatoires concernant l'organisation technique et structurelle de Vitala.

Ce document protège :

- la modularité ;
- la maintenabilité ;
- la scalabilité ;
- la séparation des responsabilités ;
- l'évolution future.

Toute IA intervenant sur l'architecture Vitala DOIT respecter ces règles.

2. Principe architectural fondamental

Vitala doit être construit comme un système modulaire.

Chaque domaine fonctionnel doit être :

- autonome ;
- isolé ;
- documenté ;
- testable ;
- évolutif.

Une fonctionnalité ne doit jamais être développée comme un bloc isolé sans respecter l'écosystème global.

3. Architecture par domaines

Vitala doit privilégier une organisation orientée fonctionnalités.

Structure recommandée :

```
```text id="7f9f4p" src/
```

features/

|— auth/

|— profile/

|— flash/

|— scan/

|— radar/

|— veille/

|— missions/

|— trust/

|— referral/

|— notifications/

|— admin/

|— analytics/

|— ai/

Chaque domaine possède sa propre logique.

---

#### # 4. Règle d'isolation des modules

Un module ne doit pas accéder directement aux fichiers internes d'un autre module.

Interdit :

```
```text
```

flash/

importe directement :

scan/components/internal/File.tsx

Autorisé :

```
flash/  
  
↓  
  
services publics  
  
↓  
  
interfaces définies
```

Chaque module communique via des contrats clairs.

5. Structure interne d'un module

Chaque feature doit suivre une organisation cohérente :

```
```text id="v6o9jd" feature-name/
```

```
|— components/
```

```
|— hooks/
```

```
|— services/
```

```
|— types/
```

```
|— utils/
```

```
|— stores/
```

```
|— tests/
```

```
|— index.ts
```

```

```

### # 6. Règle des composants

Un composant doit :

- avoir une responsabilité claire ;
- être réutilisable si nécessaire ;
- être indépendant ;
- avoir des propriétés explicites.

Un composant ne doit pas :

- contenir toute la logique métier ;
- gérer directement la base de données ;
- appeler plusieurs services complexes.

---

## # 7. Règle des services

Les services contiennent la logique externe :

- API ;
- Supabase ;
- stockage ;
- intégrations.

Un composant UI ne doit jamais gérer directement :

- requêtes SQL ;
- appels API complexes ;
- logique d'autorisation.

---

## # 8. Règle des hooks

Les hooks servent à encapsuler :

- état ;
- logique utilisateur ;
- comportements réutilisables.

Un hook ne doit pas devenir un fichier contenant toute l'application.

---

## # 9. Règle des types

Les types doivent être centralisés et cohérents.

Une IA ne doit pas créer plusieurs versions du même modèle.

Exemple interdit :

```text

User

UserType

UserModel

UserData

pour représenter la même entité sans justification.

10. Gestion des dépendances

Avant d'ajouter une dépendance externe, l'IA doit vérifier :

1. Une solution existe-t-elle déjà ?
2. La dépendance est-elle nécessaire ?
3. La sécurité est-elle acceptable ?
4. L'impact maintenance est-il acceptable ?

Une dépendance inutile est interdite.

11. Communication entre couches

Architecture obligatoire :

```text id="b1p4s6" UI

↓

Hooks

↓

Services

↓

Data Layer

↓

Database/API ```

Les couches ne doivent pas être contournées.

---

## 12. Règle Database

Le frontend ne doit jamais :

- exécuter une logique métier critique ;
  - contourner les règles backend ;
  - accéder directement à des données sensibles sans contrôle.
- 

## 13. Règle Backend

Le backend doit gérer :

- validation ;
- permissions ;
- logique métier ;
- sécurité.

Le frontend ne doit jamais être considéré comme une source fiable.

---

## 14. Règle d'évolution architecturale

Une modification architecturale importante nécessite :

1. analyse du problème ;
2. proposition ;
3. validation ;
4. documentation ;
5. implémentation.

Une IA ne doit jamais restructurer Vitala seule.

---

## 15. Règles contre la dette technique

L'IA ne doit pas :

- créer une solution temporaire sans plan ;
- copier-coller du code ;
- ignorer une mauvaise structure ;
- ajouter une dette volontaire.

Si une dette existe :

elle doit être documentée.

---

## 16. Règles de fichiers

L'IA doit :

- respecter les conventions de nommage ;
- créer les fichiers au bon endroit ;
- éviter les fichiers inutilisés.

Interdit :

- fichiers temporaires oubliés ;
  - doublons ;
  - composants abandonnés.
- 

## 17. Scalabilité obligatoire

Chaque décision doit considérer :

- augmentation du nombre d'utilisateurs ;
  - augmentation des données ;
  - augmentation des modules ;
  - multiplication des IA intervenantes.
- 

## 18. Architecture et Intelligence Artificielle

Les modules IA doivent être isolés.

Une fonctionnalité IA doit définir :

- modèle utilisé ;
  - données entrantes ;
  - données sortantes ;
  - permissions ;
  - limites ;
  - coûts.
- 

## 19. Validation architecturale

Avant validation :

[ ] Module correctement isolé

[ ] Responsabilités séparées

[ ] Dépendances justifiées

[ ] Tests présents

[ ] Documentation mise à jour

[ ] Aucun contournement architectural

---

## 20. Règle finale

L'architecture de Vitala est un actif critique.

Une IA ne doit jamais privilégier une solution rapide au détriment de l'architecture.

Toute modification doit améliorer ou préserver la structure existante.

---

## Historique

| Version | Modification                           |
|---------|----------------------------------------|
| V1.0    | Création des règles architecturales IA |

---

## Fin de AI/11-AI-ARCHITECTURE-RULES.md